

[Chap 2] Python 기초



파이썬(Python)

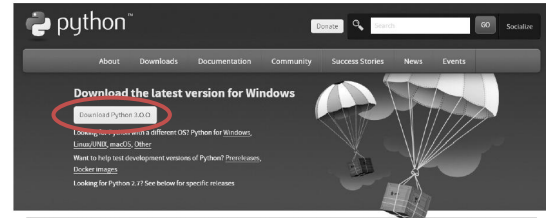
- 컴퓨터 프로그래밍 언어의 일종
 - 컴퓨터와 대화하기 위해 사용하는 여러 언어들
 - C, C++, Java, R, Python ...
- 파이썬은 문법이 간결하고 유연하며 외부 패키지 연계성과 확장성이 높은 장점이 있어 최근 널리 사용
- 특히, 텐서플로우(Tensorflow) 등 최신 분석패키지들이 파이썬에 연동 되도록 제공되는 경우가 많아 실용성이 높음

Python 설치

- 사전 필요 사항
 - ① 내가 관리자 권한을 가지고 있는 PC
 - ② 인터넷 접속
 - ③ 윈도우나 맥OS 등 OS의 기본적인 사용능력

- 파이썬 다운로드

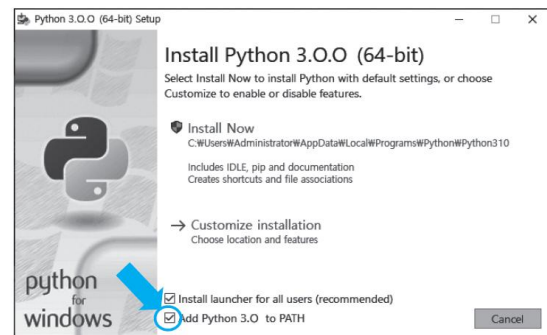
- <https://www.python.org/downloads/>



[그림 2-1] 파이썬 다운로드 페이지

- 설치 파일을 클릭하여 실행

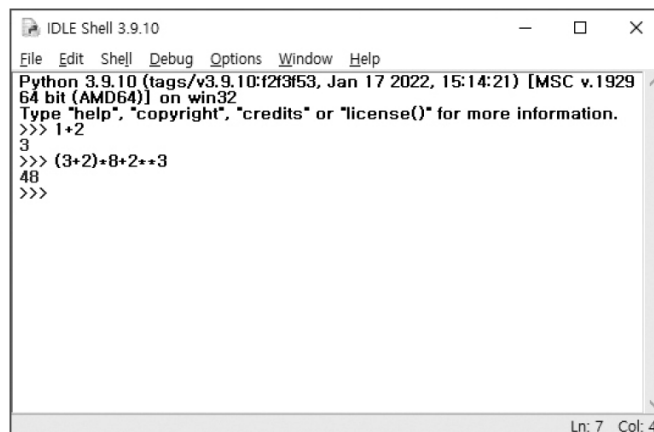
- "Add Python 3.0.0 to PATH"를 반드시 체크



3

IDLE 실행

- IDLE: 기본 설치되는 파이썬 프로그램 작성도구
- 커맨드라인 인터페이스(command-line interface)
 - IDLE shell에 수식을 입력하면 즉시 결과 제공

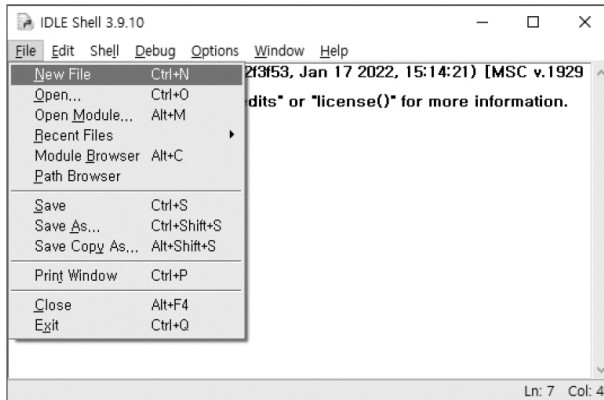


- 복잡한 로직을 구현하는 데에는 불편

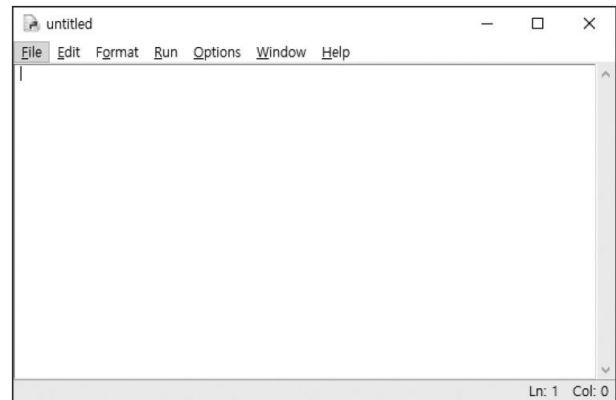
4

Script

- 대부분의 프로그래밍은 여러 줄을 작성하여 한꺼번에 실행
 - 스크립트(script) 윈도우를 열어서 내용 작성



[그림 2-7] 새 스크립트 작성 메뉴

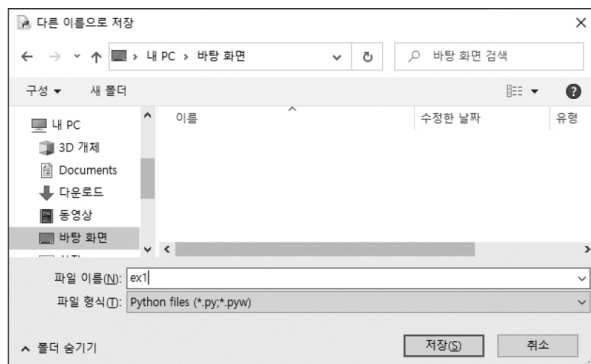
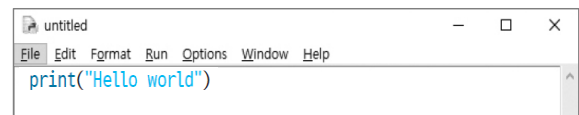


[그림 2-8] 새 스크립트 창

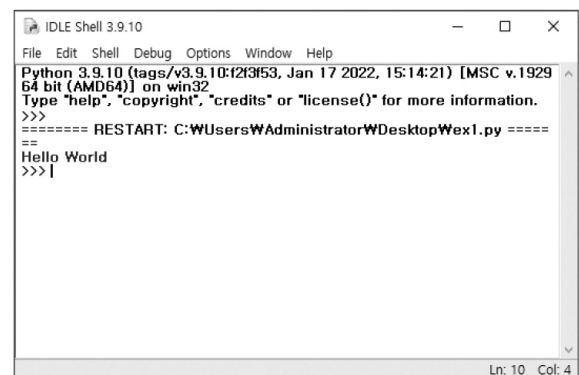
5

Hello world

- 스크립트 창에 프로그램 입력
- 저장 후 실행(F5)



[그림 2-9] 프로그램 저장 화면



[그림 2-10] Hello world 프로그램 실행 화면

- 코드 추가 후 실행

```
1 print("Hello world")
2 print("나의 첫 번째 파이썬 프로그램")
```

6

변수(variable)

- 값을 집어넣는 상자
 - `x=3` : `x`라는 상자를 만들고, 그 안에 3이라는 값을 집어넣는다

```
1 x=3
2 print(x)
3 y=5
4 print(y)
```

- 변수에는 연산을 적용할 수 있음

```
1 x=3
2 print(x)
3 y=5
4 print(y)
5 print(x+y)
6 x=7
7 print(x+y)
```

7

문자열 변수

- 변수에는 숫자 이외에도 문자열 등 다양한 자료를 넣을 수 있음
- 문자열이 들어있는 변수에 대한 `+` 연산자는 이어 붙이라는 의미

```
1 s1 = "Hello"
2 s2 = "world"
3 print(s1)
4 print(s2)
5 print(s1+s2)
6 print(s1+" "+s2)
```

8

리스트(list)

- 값의 나열을 하나로 묶은 자료
 - [] 안에 값들을 콤마(,)로 구분하여 기입

```
1 a = [10, 30, 50, 70, 90]
2 print(a)
```

- 리스트의 특정 요소는 인덱스(index)로 지정

```
1 a = [10, 30, 50, 70, 90]
2 print(a[0])
3 print(a[2])
```

- 파이썬에서 인덱스는 0부터 시작함
 - 위의 a 리스트는 0부터 4까지의 인덱스가 있으므로, a[5]는 오류 발생

```
1 a = [10, 30, 50, 70, 90]
2 print(a[0])
3 print(a[2])
4 print(a[5])
```

9

리스트의 연산과 자료형

- 리스트의 +는 이어 붙임

```
1 a = [10, 30, 50, 70, 90]
2 b = [100, 200, 300]
3 print(a+b)
```

- 리스트에 값 추가: append

```
1 a = [10, 30, 50, 70, 90]
2 print(a)
3 a.append(110)
4 print(a)
```

- 다양한 자료형이 리스트에 들어갈 수 있음

```
1 x = ['apple', 'banana', 'lemon']
2 print(x)
3 y = ['strawberry', 'grape']
4 print(x+y)
```

조건문(if)

- if 라는 키워드 다음에 체크할 조건이 나오고, 콜론(:) 기호
 - 그 다음줄부터는 약간 들여쓰기가 되어 있고, 조건이 만족될 때 실행할 명령문들 (코드 블록, code block)이 나옴
 - 조건문의 코드 블록은 if 블록이라고 함

문법 if 문

if (조건):
(조건이 만족되면 실행할 명령 블록)

- 나이를 입력받아 20세 이상일 때에만 주류 메뉴 보이게 하기

```
1 age=int(input('나이를 입력하세요 : '))
2 print('<Drink Menu>')
3 print('1. 콜라')
4 print('2. 사이다')
5 if age>=20:
6     print('3. 맥주')
7     print('4. 소주')
8 print('메뉴를 선택해 주세요')
```

11

if.. else 문

- if와 else의 결합 구조

문법 if ... else 문

if (조건):
(조건이 만족되면 실행할 명령 블록)
else:
(조건이 만족되지 않을 때 실행할 명령 블록)

- 나이가 20세 이상이면 주류 메뉴를 표시하고, 그렇지 않으면 오렌지주스를 메뉴에 나타내기

예제 ex2-1.py

```
1 age=int(input('나이를 입력하세요 : '))
2 print('<Drink Menu>')
3 print('1. 콜라')
4 print('2. 사이다')
5 if age>=20:
6     print('3. 맥주')
7     print('4. 소주')
8 else:
9     print('3. 오렌지주스')
10 print('메뉴를 선택해 주세요')
```

12

반복문: while 문

- 반복문(loop)은 프로그래밍 언어에서 필수적인 기능
 - while과 for가 대표적
- while: 특정 조건을 만족하는 동안 코드 블록을 반복

문법 while 문
while (조건):
(조건이 만족되면 실행할 명령 블록)

- 1부터 10까지 출력하고 마지막에 'end!' 출력하기

```
1 i = 1
2 while i <= 10:
3     print(i)
4     i = i + 1
5 print('end!')
```

13

무한루프

- 무한 반복하다가 특정 조건 만족 시 빠져나오는 형태는 자주 사용됨
- while True: 는 항상 만족하는 조건(True)이므로 코드블록을 무한 반복
 - 반복을 중단하고 while 루프를 빠져나오려면 break문 사용
- 1부터 10까지 출력하고 마지막에 'end!' 출력하기 (무한루프 사용)

```
1 i = 1
2 while True:
3     print(i)
4     if i==10: break
5     i = i + 1
6 print('end!')
```

14

무한루프 (계속)

- [예제 2-2] 피보나치(Fibonacci) 수열로 황금비 계산

```
예제 ex2-2.py
1 # Fibonacci 수열로 황금비 계산
2 a = 1
3 b = 1
4 print(a)
5 print(b)
6 while True:
7     c = a + b
8     print(c, c/b)
9     if c > 100000: break
10    a = b
11    b = c
12 print('프로그램 끝')
```

15

반복문: for

- 리스트의 값을 하나씩 꺼내 오면서 루프 생성

```
문법 for 문
for (변수이름) in (리스트):
    (리스트 안에 있는 각 요소에 대해 실행할 명령 블록)
```

- for 문 예제

```
1 fruits = ['apple', 'banana', 'lemon']
2 for x in fruits:
3     print(x)
4 print('프로그램 끝!')
```

- 1에서 100까지 합계 계산

```
1 s = 0
2 for i in range(101):
3     print(i)
4     s = s + i
5 print('합계는',s)
```

16

중첩 for문

- 리스트1의 요소를 하나 꺼내서 변수1에 넣은 다음, 리스트2의 각 요소에 대해 코드 블록을 반복
 - 리스트2의 요소 전체에 대해 반복이 완료되면, 리스트1의 두 번째 요소를 꺼내서 다시 리스트2의 각 요소에 대해 반복

```
1 for (변수1) in (리스트1):
2     for (변수2) in (리스트2):
3         (반복 실행 코드 블록)
```

- 구구단 출력 예제

```
1 for i in range(2,10):
2     for j in range(1,10):
3         print(i,'*',j,'=',i*j)
4     print()
```

17

함수(function)

- 모듈화 프로그래밍(modular programming)
 - 대입문, 출력문, 조건문, 반복문만으로도 상상 가능한 모든 프로그램을 다 만들어낼 수 있으나, 매우 난해하고 관리가 어려워짐
 - 이를 해결하기 위해 큰 프로그램은 프로그램을 특정 모듈(module) 단위로 나누어서 계층적으로 구현
- 함수는 입력값을 넘겨주면, 해당 입력값을 이용해서 어떤 처리를 수행하고 그 결과값을 내보내주는 블랙박스
 - 함수에 넘겨주는 입력값: 인자(argument)
 - 함수의 처리 결과로 내보내주는 결과값: 반환값 또는 리턴값(return value)
- 어떤 값을 넘겨주면 100배를 만들어 리턴하는 함수

```
1 def multiplier(a):
2     a = a*100
3     return a
```

- 이 함수를 호출하기

```
4 print('출력결과 : ',multiplier(2))
```

18

함수와 변수의 유효범위

- 함수 안에서 사용된 변수는 이름이 같더라도 함수 바깥에 있는 변수와는 전혀 다른 변수임
 - 함수 안에서 정의된 변수는 함수 안에서만 생명력을 가짐

```
1 def multiplier(a, b):  
2     y = a+b+a*b  
3     return y  
4 y=1  
5 a=10  
6 b=20  
7 print(multiplier(a,b))  
8 print('hello y=', y)
```

- 함수 내에서 정의된 변수의 유효 범위를 함수 안으로만 한정함으로써, 함수를 호출해서 사용할 때 그 안에 무슨 이름의 변수를 사용했는지 몰라도 되는, 즉 블랙박스로 취급한 구현이 가능하게 됨